Gym prediction using Logistic regression

```
import pandas as pd

# Load the dataset
df = pd.read_csv('gym_decision_dataset.csv')

# Display the first few rows
print("First 5 rows of the dataset:")
print(df.head())

# Check data types and for missing values
print("\nData Info:")
print(df.info())
```

```
First 5 rows of the dataset:
   motivation_level  distance_to_gym_km weather  netflix_continue day_of_week  \
0                 7            1.995296  cloudy                 0         Sat
1                 4           13.587017   sunny                 0         Thu
2                 8            7.826159  cloudy                 0         Thu
3                 5           12.483633    snow                 0         Mon
4                 7            5.140719   sunny                 0         Wed

   went_to_gym
0            1
1            1
2            1
3            0
4            1

Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   motivation_level    500 non-null    int64
 1   distance_to_gym_km  500 non-null    float64
 2   weather             500 non-null    object
 3   netflix_continue    500 non-null    int64
 4   day_of_week         500 non-null    object
 5   went_to_gym         500 non-null    int64
dtypes: float64(1), int64(3), object(2)
memory usage: 23.6+ KB
None
```

Data Preprocessing:

a.handling categorical features

```
# 'day_of_week' and 'weather' are the columns to encode
df_encoded = pd.get_dummies(df, columns=['weather', 'day_of_week'], drop_first=True)

print("\nEncoded Data Info:")
print(df_encoded.head())
```

```
Encoded Data Info:
   motivation_level  distance_to_gym_km  netflix_continue  went_to_gym  \
0                 7            1.995296                 0            1
1                 4           13.587017                 0            1
2                 8            7.826159                 0            1
3                 5           12.483633                 0            0
4                 7            5.140719                 0            1

   weather_rainy  weather_snow  weather_sunny  day_of_week_Mon  \
0          False         False          False            False
1          False         False           True            False
2          False         False          False            False
3          False          True          False             True
4          False         False           True            False

   day_of_week_Sat  day_of_week_Sun  day_of_week_Thu  day_of_week_Tue  \
0             True            False            False            False
1            False            False             True            False
2            False            False             True            False
3            False            False            False            False
4            False            False            False            False

   day_of_week_Wed
0            False
```

```
1            False
2            False
3            False
4             True
```

b.separating features and target

```
# Separate features (X) and target (y)
# 'went_to_gym' is the target variable
X = df_encoded.drop('went_to_gym', axis=1)
y = df_encoded['went_to_gym']

# Check the shapes of X and y
print(f"\nShape of X (Features): {X.shape}")
print(f"Shape of y (Target): {y.shape}")
```

```
Shape of X (Features): (500, 12)
Shape of y (Target): (500,)
```

c.separating train and test set

```
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nTraining set size: {len(X_train)} samples")
print(f"Testing set size: {len(X_test)} samples")
```

```
Training set size: 400 samples
Testing set size: 100 samples
```

3. Training the Logistic Regression Model

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model
# Using max_iter for convergence, though default is often fine
model = LogisticRegression(random_state=42, max_iter=1000)

# Train the model
model.fit(X_train, y_train)

print("\nLogistic Regression model training complete.")
```

```
Logistic Regression model training complete.
```

4.Model Evaluation

```
from sklearn.metrics import accuracy_score, classification_report

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"\nModel Accuracy on Test Set: {accuracy:.4f}")
print("\nClassification Report:\n", report)
```

```
Model Accuracy on Test Set: 0.8100

Classification Report:
               precision    recall  f1-score   support

           0       0.81      0.66      0.72        38
           1       0.81      0.90      0.85        62

    accuracy                           0.81       100
   macro avg       0.81      0.78      0.79       100
weighted avg       0.81      0.81      0.81       100
```

Interactive Prediction Tool

```
import pandas as pd
import numpy as np

def get_user_input(feature_columns):
    """
    Interactively gathers user inputs for gym attendance prediction.
    """
    print("\n" + "="*50)
    print("GYM ATTENDANCE PREDICTOR")
    print("Please answer a few questions about your day:")
    print("="*50)

    # 1. Motivation Level
    while True:
        try:
            motivation = int(input("1. On a scale of 1 to 10, what is your **Motivation Level** right now? (1=low, 10=high): "
            if 1 <= motivation <= 10:
                break
            else:
                print("Please enter a number between 1 and 10.")
        except ValueError:
            print("Invalid input. Please enter a whole number.")

    # 2. Distance to Gym
    while True:
        try:
            distance = float(input("2. What is the **Distance to your gym** in kilometers (e.g., 5.5): "))
            if distance >= 0:
                break
            else:
                print("Distance cannot be negative.")
        except ValueError:
            print("Invalid input. Please enter a numerical value.")

    # 3. Netflix Binge
    while True:
        netflix_raw = input("3. Are you currently in the middle of a **Netflix binge**? (yes/no): ").strip().lower()
        if netflix_raw in ['yes', 'y']:
            netflix = 1
            break
        elif netflix_raw in ['no', 'n']:
            netflix = 0
            break
        else:
            print("Invalid input. Please enter 'yes' or 'no'.")

    # 4. Weather
    valid_weather = ['cloudy', 'rainy', 'snow', 'sunny']
    while True:
        weather = input(f"4. What is the **Weather** right now? ({', '.join(valid_weather)}): ").strip().lower()
        if weather in valid_weather:
            break
        else:
            print(f"Invalid input. Please enter one of: {', '.join(valid_weather)}.")

    # 5. Day of the Week
    valid_days = ['mon', 'tue', 'wed', 'thu', 'fri', 'sat', 'sun']
    while True:
        day = input(f"5. What is the **Day of the Week**? ({', '.join([d.capitalize() for d in valid_days])}): ").strip().lowe
        if day in valid_days:
            break
        else:
            print(f"Invalid input. Please enter a valid day (e.g., Mon, Tue, etc.).")


    # --- Create the Model-Ready Input DataFrame ---

    # 1. Initialize a DataFrame with all feature columns set to 0
    input_df = pd.DataFrame(0, index=[0], columns=feature_columns)

    # 2. Populate the numerical columns
    input_df['motivation_level'] = motivation
    input_df['distance_to_gym_km'] = distance
    input_df['netflix_continue'] = netflix

    # 3. Set the correct one-hot encoded columns to 1 (handling the 'drop_first' baseline)

    # Weather Encoding (assuming 'cloudy' was dropped/is the baseline)
    weather_col = f'weather_{weather}'
```

```
    if weather_col in feature_columns:
        input_df[weather_col] = 1

    # Day of Week Encoding (Sunday is likely the dropped/baseline category)
    day_col = f'day_of_week_{day.capitalize()[:3]}' # e.g., 'day_of_week_Mon'
    if day_col in feature_columns:
        input_df[day_col] = 1

    return input_df, day.capitalize()[:3]


def predict_and_report(model, X_train_columns):
    """
    Gets user input, predicts gym attendance, and prints the result.
    """
    # Get the user's data prepared for the model
    input_df, day_input = get_user_input(X_train_columns)

    # Make the prediction
    prediction = model.predict(input_df)[0]

    # Get the probability of going to the gym (class 1)
    probability = model.predict_proba(input_df)[0][1]

    # Interpret the result
    print("\n" + "="*50)
    print("MODEL PREDICTION")
    print("="*50)

    if prediction == 1:
        print(f"Based on your input, the model predicts a high chance you will **GO TO THE GYM** today ({day_input})! 💪")
    else:
        print(f"Based on your input, the model predicts you will **SKIP THE GYM** today ({day_input}). 🛋️")

    print(f"Probability of going to the gym: **{probability:.2f}** (or {probability*100:.0f}%)")
    print("="*50)


# --- Execution ---

# Run the interactive predictor using the model and features from the previous step
predict_and_report(model, X_train.columns)
```

```
==================================================
GYM ATTENDANCE PREDICTOR
Please answer a few questions about your day:
==================================================
1. On a scale of 1 to 10, what is your **Motivation Level** right now? (1=low, 10=high): 7
2. What is the **Distance to your gym** in kilometers (e.g., 5.5): 5
3. Are you currently in the middle of a **Netflix binge**? (yes/no): no
4. What is the **Weather** right now? (cloudy, rainy, snow, sunny): rainy
5. What is the **Day of the Week**? (Mon, Tue, Wed, Thu, Fri, Sat, Sun): sat

==================================================
MODEL PREDICTION
==================================================
Based on your input, the model predicts a high chance you will **GO TO THE GYM** today (Sat)! 💪
Probability of going to the gym: **0.91** (or 91%)
==================================================
```